8.Implementation of Boosting Techniques (AdaBoost) and Analyze Performance

Aim:

To implement the **AdaBoost** boosting technique, which combines several weak learners (decision stumps) into a strong classifier, and to analyze its performance using **accuracy, precision, recall, F1-score, ROC-AUC** and **cross-validation**.

Algorithm :

A. AdaBoost Implementation (binary, labels y ∈ {−1, +1})

Input: Training set (x_1, y_1), ..., (x_N, y_N); number of rounds **M**; weak learner (decision stump)

Step 1: Initialise the sample weights: $w_i = 1/N$ for all i.

Step 2: For **m = 1 to M**:

1. Train the weak learner $h_m(x)$ on the training data using the weights w.
2. Compute the weighted error:

$$\varepsilon_m = \Sigma\, w_i \cdot [h_m(x_i) \neq y_i] \,/\, \Sigma\, w_i$$

3. Compute the learner weight:

$$\alpha_m = \tfrac{1}{2} \cdot \ln((1 - \varepsilon_m) / \varepsilon_m)$$

4. Update the sample weights:

$$w_i \leftarrow w_i \cdot \exp(-\alpha_m \cdot y_i \cdot h_m(x_i))$$

(misclassified samples receive higher weights)

5. Normalise the weights so that $\Sigma\, w_i = 1$.

Step 3: Output the final strong classifier:

$$H(x) = \text{sign}(\Sigma_m\, \alpha_m \cdot h_m(x))$$

B. Performance Analysis Procedure

Step 1: Load the dataset and split it into **training and test sets** (stratified).

Step 2: Train a **single weak learner** (decision stump) as the baseline.

Step 3: Train the **AdaBoost** model with M estimators.

Step 4: Predict on the test set and compute **accuracy, precision, recall, F1-score, ROC-AUC** and the **confusion matrix**.

Step 5: Perform **k-fold cross-validation** and record the **training time**.

Step 6: Plot **error vs. number of boosting rounds** and the **ROC curve**.

Step 7: Compare AdaBoost with the baseline and **conclude** whether boosting improved performance.

Program,Result&Graph:

